



F6 简易输入输出 — Logisim 从零实现完整指南

适用读者：刚安装好 Logisim，对软件操作不熟悉，需要从零搭建 sCPU 并添加 I/O 功能。

第一部分：Logisim 快速入门

1.1 下载与安装

- 下载地址：<https://github.com/logisim-evolution/logisim-evolution/releases>
- Windows 用户下载 `.msi` 安装包，双击安装即可
- 需要 Java 运行环境，如果没有请先安装 Java 21+

1.2 界面概览

打开 Logisim 后你会看到：



区域	功能
工具栏	箭头（选择/编辑/连线）、手指（Poke，用于点击输入引脚切换 0/1）
左侧元件库	按类别列出所有可用元件，点击后在画布上放置
画布	主编辑区域
左下属性面板	选中元件后在此修改参数（位宽、方向、数量等）
左上电路选择器	切换 <code>main</code> 电路和子电路

1.3 核心操作

操作	方法
放置元件	在元件库中点选元件 → 在画布上点击放置
连线	用 箭头工具 ，鼠标移到元件引脚处（出现绿色小圆点），拖拽到目标引脚

操作	方法
修改属性	选中元件，在 左下角属性面板 修改
旋转元件	选中元件，属性面板中改 Facing (North/South/East/West)
测试电路	切换到 手指工具 (Poke) ，点击输入引脚切换 0/1
创建子电路	菜单 Project → Add Circuit ，输入名称
保存	Ctrl+S (Logisim 不会自动保存 ，务必勤保存！)
开始/停止仿真	Ctrl+K 或菜单 Simulate → Ticks Enabled
调整仿真速度	菜单 Simulate → Tick Frequency
复位	菜单 Simulate → Reset Simulation

1.4 导线颜色含义（调试必备）

颜色	含义	原因
深绿色	1位信号 = 0	正常
亮绿色	1位信号 = 1	正常
黑色	多位信号	正常（看不出具体值，用 Probe 查看）
红色	冲突	多个输出驱动同一条线且值不同 → 去掉多余驱动
蓝色	悬空	没有任何驱动 → 接到信号源
橙色	位宽不匹配	比如 4 位线接到了 8 位端口 → 统一位宽

1.5 常用元件速查

Wiring（连线类）

元件	用途	关键属性
Pin（输入引脚）	引入外部信号，Pokable	Data Bits、Label

元件	用途	关键属性
Output Pin（输出引脚）	暴露信号观察	Data Bits
Probe（探针）	查看任意节点值	无，直接拖到线上
Tunnel（隧道）	隐形连线，同名即连通	Label（名称要一致）
Splitter（分离器）	拆分/合并多位信号	Bit Width In/Out、Fan Out
Constant（常量）	固定值（如 0、1）	Data Bits、Value
Ground/Power	接地/接电源（=常量0/1）	—

Plexers（选择器类）

元件	用途	关键属性
Multiplexer（多路选择器）	N选1，sel 端选择哪路输入通过	Data Bits、Select Bits
Demultiplexer（多路分配器）	1选N，sel 决定输出到哪路	Data Bits、Select Bits
Decoder（译码器）	N位输入 → 2^N 位独热码输出	输出位数

Arithmetic（运算类）

元件	用途	关键属性
Adder（加法器）	$A + B$	Data Bits
Subtractor（减法器）	$A - B$	Data Bits
Comparator（比较器）	比较 A 和 B	Data Bits、输出有 =、≠、>、< 等
Shifter（移位器）	左移/右移	Data Bits

Memory（存储类）

元件	用途	关键属性
D Flip-Flop	1位触发器	—

元件	用途	关键属性
Register	多位寄存器，时钟上升沿锁存	Data Bits、Label
RAM	可读可写存储器	Data Bits、Address Bit Width
ROM	只读存储器	Data Bits、Address Bit Width

ROM 编辑内容：右键 ROM → **Edit Contents**，以十六进制填写每地址的存储值。

Input/Output

元件	用途	关键属性
Button	手动按压输入	—
LED	单颗指示灯	—
LED Bar	多段 LED 条	Input Format=One Wire, Segments=8
7-Segment Display	数码管	—
DIP Switch	多位拨码开关	Data Bits

1.6 子电路 (Subcircuit)

子电路相当于编程中的函数，将复杂电路封装成可复用的模块。

创建子电路：

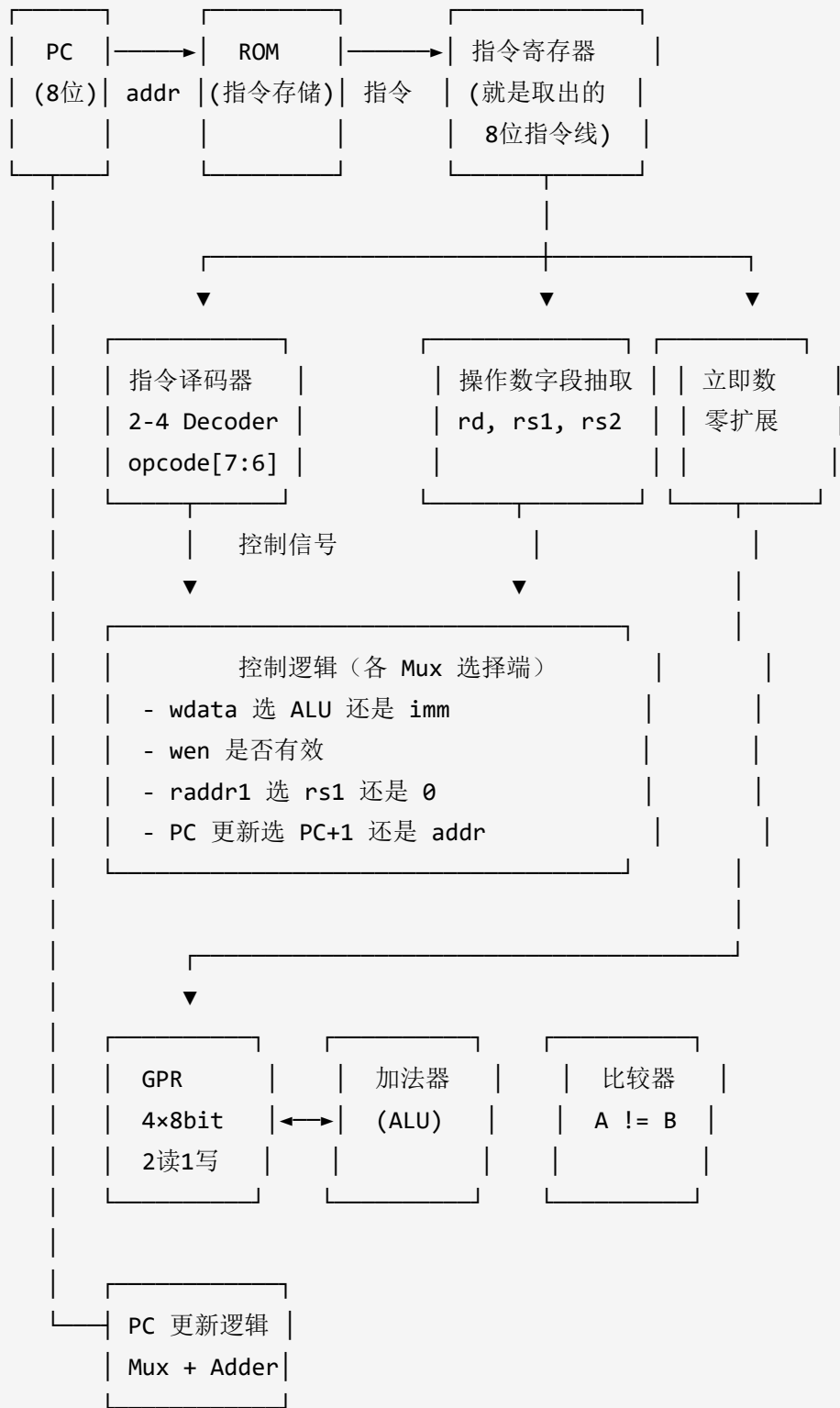
1. 菜单 **Project** → **Add Circuit**，命名（必须以字母开头）
2. 左侧出现新的子电路名，双击进入编辑
3. 在子电路中用 **Pin** 元件做输入输出端口，用 **Tunnel** 对外连接
4. 编辑完双击 **main** 回到主电路
5. 在左侧**单击**子电路名，再在画布上点击即可放置实例

查看子电路内部：右键已放置的实例 → **View [名称]**

第二部分：回顾 sCPU 已有架构（F5 成果）

在开始 F6 之前，确保你已经完成 F5 的 sCPU 实现。以下快速回顾关键点：

sCPU 架构（F5 完成时）：



F5 关键数据：

- PC: 8 位寄存器, 初值 0
- GPR: 4 个 8 位寄存器 (R0-R3) , 2 个读口 + 1 个写口
- ROM: 深度至少 $\geq$ 程序指令条数 (如 8 条) , 宽度 8 位
- 指令译码器: 2-4 Decoder, 输出 4 根控制线
- 2 选 1 Mux (GPR wdata) : 选 ALU 结果或立即数
- 2 选 1 Mux (PC 更新) : 选 PC+1 或绝对地址
- 比较器: 输出 $R[0] \neq R[rs2]$

第三部分：F6 电路修改 — 增强 li 指令

3.1 指令格式变化

```
F5: | 10 | rd |   imm   | R[rd] = zero_ext(imm)

F6: | 10 | rd |  s  | imm | R[rd] = imm << (s << 1)
```

- `s` 字段在 bit 3-2, 控制 `imm` (bit 1-0) 放置到 8 位中的哪一段
- `s << 1` 即 $s \times 2$, 是移位位数

3.2 Logisim 操作步骤

等价理解：4 选 1

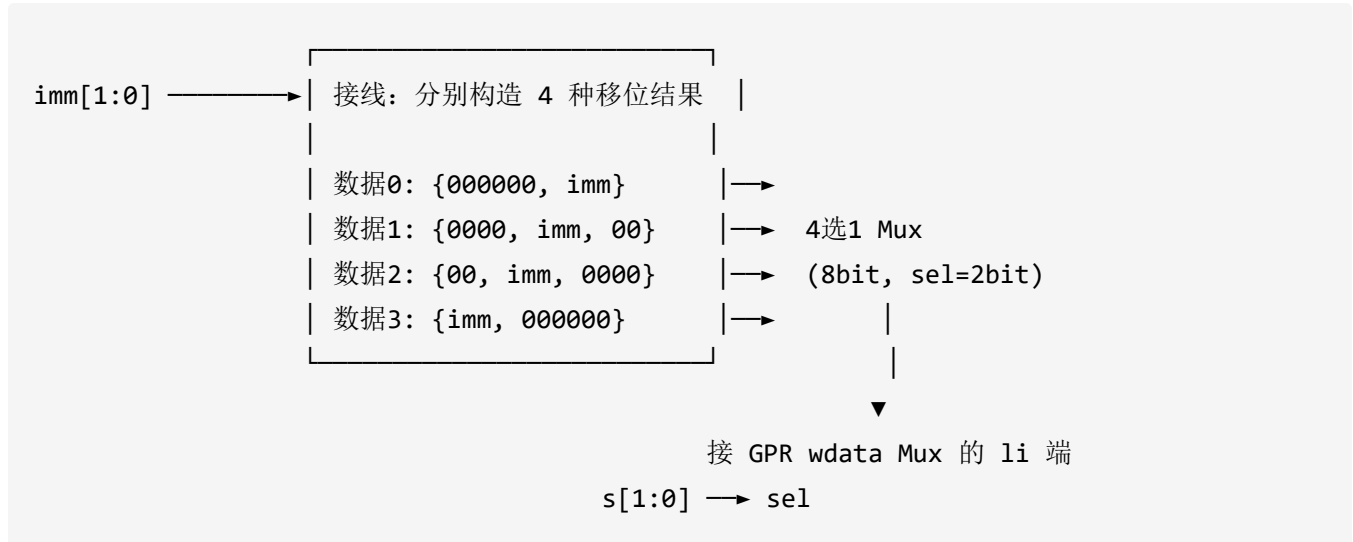
`s` 有 4 种取值, 对应 4 种 8 位立即数结果:

s	8 位结果	十六进制示例 (imm=11)
00	000000XX	00000011 = 0x03
01	0000XX00	00001100 = 0x0C
10	00XX0000	00110000 = 0x30
11	XX000000	11000000 = 0xC0

电路改动

原来：imm (2位) → Bit Extender (零扩展到 8 位) → GPR wdata Mux 的 li 输入端

现在：插入一个 **4 选 1 Mux** (Multiplexer, 8 位宽, 2 位选择端)



具体做法 (推荐新手做法)：

1. 放置一个 **Constant** (Data Bits = 6, Value = 0x00)，用于提供 6 个 0
2. 放置一个 **Splitter**:
 - Fan Out = 2
 - Bit 0~1 对应 imm[0] 和 imm[1]
 - Bit 2~7 接地 (或接上面那个 6 位常量 0)
3. 同上方法构造 4 种排布，分别作为 Mux 的 4 个数据输入
4. 或者：放置 4 个 **Constant** (各 8 位)，直接填好值，然后接 Mux

更简单的做法：只用一个 8 位常量 (或拼接)，在 Mux 属性里无法直接设不同值——所以 4 个 Constant 各接 Mux 的 4 个输入端是最直观的。缺点是 immediate 值变了就要改 4 个常量。**实用做法**是用 Bit Extender (2→8) + Shifter 做动态移位，下面介绍：

进阶做法 (推荐，支持动态 imm)

1. 放置 **Bit Extender** (Wiring 分类下可能是 "Bit Width" 相关)：
 - Input Width = 2, Output Width = 8
 - Extension Type = Zero
 - 输入端接指令的 imm[1:0] 位

- 输出得到一个 8 位值 `{000000, imm}`

2. 放置 **Shifter** (Arithmetic 分类) :

- Data Bits = 8
- 数据输入端接 Bit Extender 的输出
- Shift 输入端: 需要 `s * 2`。用一个 2 位乘法器或直接:
 - `s[1:0] →` 接一个 2 选 1 Mux (或逻辑), `s=00→0, 01→2, 10→4, 11→6`
- 或者更简单: 直接把 `s` 的 2 位接到一个 **自定义拼接**:
 - `s=00` 时接 0
 - 最偷懒: `s` 本身左移 1 位就是 `s×2`。用 Splitter 把 `{s[1:0], 1'b0}` 拼成 3 位接到 Shifter 的 Shift 端 (前提是 Shifter 的 Shift 端支持 3 位)

实际上 Logisim 的 Shifter 的 Shift 端接受多位输入。直接把 `{s, 0}` 接过去就行 (用一个 Splitter 拼位: `s[1], s[0], 0 = 3位 = s×2`)。

3.3 验证

在 ROM 中写入测试指令:

地址	指令	机器码 (二进制)	期望结果
0	<code>li r0, s=00, imm=11</code>	<code>10_00_00_11 = 0x83</code>	<code>r0 = 0x03</code>
1	<code>li r0, s=11, imm=10</code>	<code>10_00_11_10 = 0x8E</code>	<code>r0 = 0x80</code>

用 Probe 观察 GPR 写数据 Mux 的输出和 `r0` 的值。

第四部分: F6 电路修改 — 增强 `bner0` 指令

4.1 指令格式变化

F5: | 11 | addr | rs2 | if (`R[0] != R[rs2]`) PC = addr

F6: | 11 | offset | rs2 | if (`R[0] != R[rs2]`) PC = PC + `sign_ext(offset)`

- `offset` 只有 4 位 (bit 5-2), 是**有符号数** (补码), 范围 -8 ~ +7

- 目标地址 = $PC + \text{sign_ext}(\text{offset})$ (相对跳转)
- 不再是绝对地址!

4.2 Logisim 操作步骤

步骤 1: 分离 offset 字段

用 **Splitter** 从 8 位指令中分离出 $\text{offset}[3:0]$ (即指令的 bit 5-2) :

```
指令[7:0] → Splitter →  
bit 7-6: opcode  
bit 5-4: 上半 → offset 的高 2 位  
bit 3-2: 下半 → offset 的低 2 位  
bit 1-0: rs2
```

将 offset 的 4 位组合成一条 4 位线。

步骤 2: 符号扩展 (Sign Extend)

4 位 offset → 8 位:

```
 $\text{sign\_ext}(\text{offset}) = \{\text{offset}[3], \text{offset}[3], \text{offset}[3], \text{offset}[3], \text{offset}[3:0]\}$ 
```

方法: 在 Logisim 中放置一个 **Bit Extender**:

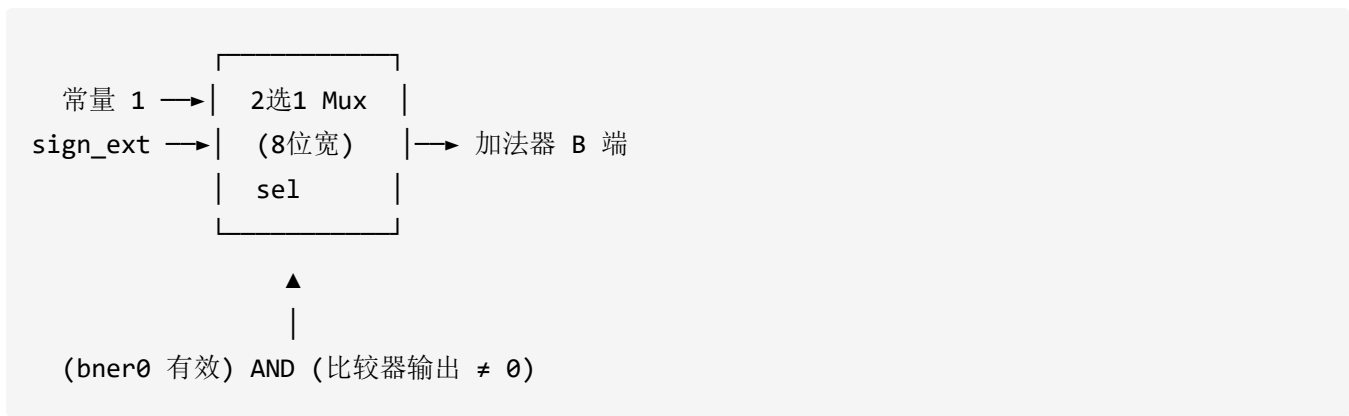
- Input Width = 4
- Output Width = 8
- Extension Type = **Sign Extend**

Bit Extender 的输入端接 offset 的 4 位线, 输出就是符号扩展后的 8 位值。

步骤 3: 修改 PC 加法器

F5 中 PC 加法器的一个输入端固定接常量 **1** (始终算 $PC+1$) 。

现在改为:



- sel = 0 → 选 1 (正常 PC+1)
- sel = 1 → 选 sign_ext(offset) (bner0 跳转时)

连接：将 bner0 对应的译码器输出（独热码线）与比较器的"不相等"输出用 **AND 门**连接，输出接 Mux 的 sel 端。

注意：如果不是 bner0 指令，或者 bner0 但比较相等，sel 都应 = 0，选 PC+1。

4.3 更新后的控制信号表

指令	GPR wen	GPR wdata 来源	GPR raddr1	PC 加法器 B 端
add (00)	1	ALU 结果	rs1	1
io (01)	i/o==0 时=1	见第五部分	rs1 或 rd	1
li (10)	1	li 移位后 imm	无关	1
bner0 (11)	0	无关	0	1 或 sign_ext(offset)

4.4 验证

在 ROM 中测试跳转：

```

# 地址 0: li r0, 00_11    # r0 = 3
# 地址 1: li r1, 00_01    # r1 = 1
# 地址 2: add r1, r1, r1   # r1 = 2
# 地址 3: bner0 -2, r0     # r0=3, r1=2, 不相等 → 跳回地址 2
  
```

机器码（假设你的 bner0 偏移计算为 PC 当前地址 + offset）：

- 偏移 -2 = 补码 1110

如果循环正确，r1 会无限递增。

第五部分：F6 电路修改 — 添加 io 指令（重点）

5.1 指令格式

```

  7  6  5  4  3    2  1  0
+---+---+---+---+
| 01 | rd | i/o | idx |
+---+---+---+---+
```

字段	位	含义
opcode	7-6	01 = io 指令
rd	5-4	目标/源寄存器号
i/o	2	0 = Input（从设备读入 R[rd]），1 = Output（将 R[rd] 输出到设备）
idx	1-0	设备编号（可选 0-3 号设备）

5.2 指令译码扩展

F5 中已使用 2-4 Decoder 对 opcode[7:6] 译码：

opcode	Decoder 输出线	对应指令
00	第 0 线（最低位）	add
01	第 1 线	io（新增，之前未用）
10	第 2 线	li
11	第 3 线（最高位）	bner0

不需要添加任何新元件！第 1 根输出线天然就是 `io_valid` 信号。

5.3 分离 io 指令字段

从 8 位指令中用 **Splitter** 抽取：

- `rd` (bit 5-4) → 和 `add/li` 共用即可
- `i/o` (bit 2) → 需要单独抽出来
- `idx` (bit 1-0) → 设备编号

5.4 输出通路：让 LED Bar 亮起来

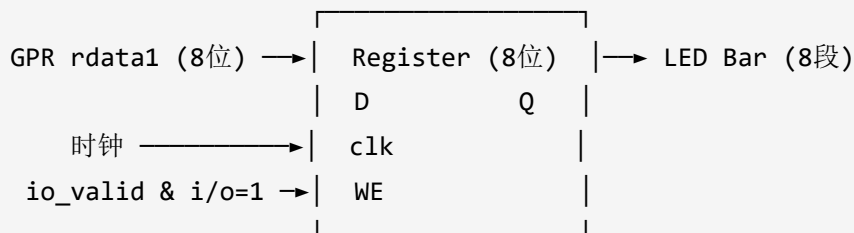
5.4.1 放置 LED Bar

1. 在元件库 → **Input/Output** → 找到 **LED Bar**
2. 点击后放置在画布上
3. 选中 LED Bar，在左下角属性面板修改：
 - **Input Format** → 选 **One Wire** (不是 `Separate`, 不是 `Bus`)
 - **Segments** → 8
 - **On Color** → 选红色或其他你喜欢的颜色

5.4.2 添加输出锁存器

LED Bar 是组合逻辑元件，输入信号一变化 LED 就立即变化。为了让 LED 只在 `io` 输出指令执行后更新，加一个 **Register** 锁存：

1. 放置一个 **Register** (Memory 分类)：
 - **Data Bits** → 8
 - **Label** → 输出锁存器 (可选，便于识别)
 - **WE (Write Enable)** → 接 `io_valid AND (i/o == 1)`
2. 寄存器 `D` (数据输入) → 接 **GPR rdata1** (经过适当选择，确保读出 `R[rd]`)
3. 寄存器 `Q` (数据输出) → 接 **LED Bar 的输入端**
4. 寄存器 `clk` → 和 PC、GPR 共用同一个时钟信号



时钟说明：如果你的 sCPU 已经有一个全局时钟（所有寄存器共用），直接连上即可。如果没有统一时钟，需要在 Simulation 中 Ticks Enabled (Ctrl+K)，并在电路中放置 **Clock** 元件（Wiring 分类）连接所有寄存器的 clk 端。

5.5 输入通路：从设备读取数据

5.5.1 模拟输入设备

在 Logisim 中，可以用以下方式模拟输入设备：

方案 A（最简单，推荐新手）：用 **Constant** 元件

- 放置一个 Constant (Wiring 分类)，Data Bits = 8, Value 随意填（如 0x55）
- 当 io 指令 i/o=0 时，将此常量值写入 GPR
- 测试时修改 Value 即可改变“输入值”

方案 B（交互式）：用 **DIP Switch** 元件

- Input/Output → DIP Switch, Data Bits = 8
- 运行时手动拨动开关提供输入

方案 C（多设备）：支持 idx 选择

- 放置 4 个 Constant（各代表 dev[0] ~ dev[3]）
- 用一个 **4 选 1 Mux**，sel = idx[1:0]，数据端分别接 4 个常量
- Mux 输出 = 选中的设备数据

5.5.2 将设备数据引入 GPR 写通路

当前 GPR 写数据 Mux 有 2 个来源：ALU 结果（add 时选）、li 移位后的立即数（li 时选）。

现在需要加入第 3 个来源：**设备输入数据**。

做法：将原来的 2 选 1 Mux 替换为 **4 选 1 Mux** (Data Bits = 8, Select Bits = 2) , 或级联两个 2 选 1 Mux。

推荐：级联两个 2 选 1 Mux (思路更清晰)

第1级 Mux (选择「运算类」还是「非运算类」)：

`sel = add_valid` (来自译码器第0线)

数据0 = 设备/li 的结果 (来自第2级 Mux)

数据1 = ALU 加法器输出

第2级 Mux (选择「li 立即数」还是「设备输入」)：

`sel = io_valid AND (i/o == 0)`

数据0 = li 移位后的 8 位立即数

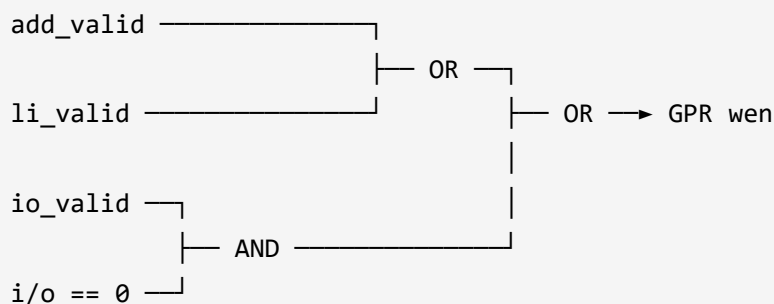
数据1 = 设备输入数据 (Constant / DIP Switch / Mux 输出)

最终 GPR wdata = 第 1 级 Mux 的输出。

5.6 修改 GPR 写使能 (wen)

```
wen = add_valid OR li_valid OR (io_valid AND i/o == 0)
```

用一个 **OR 门** (3 输入, 或级联两个 2 输入 OR 门)：



注意：i/o==0 的判断：i/o 只有 1 位，用 **NOT 门**取反即可表示 i/o == 0。

5.7 修改 GPR 读地址 (raddr1)

F5 中 `raddr1` 是一个 2 选 1 Mux 的输入：

- 数据 0 = `rs1` 字段 (指令 bit 3-2) → add 时使用
- 数据 1 = 常量 `0` (即 `R[0]`) → bner0 时使用
- 原来 `sel = bner0_valid`

现在 io 输出时也需要读出 `R[rd]` :

将 `raddr1` 的 2 选 1 Mux 改为 **4 选 1 Mux** (Select Bits = 2) , 或增加一级 Mux:

```
raddr1 Mux (8位宽, 4选1):
  数据0 = rs1 字段      (add 用)
  数据1 = 0             (bner0 用)
  数据2 = rd 字段      (io 用)
  数据3 = 无关
  sel[0] = bner0_valid OR (io_valid AND i/o==1)
  sel[1] = ...
```

或者最简单的方式——用级联 Mux:

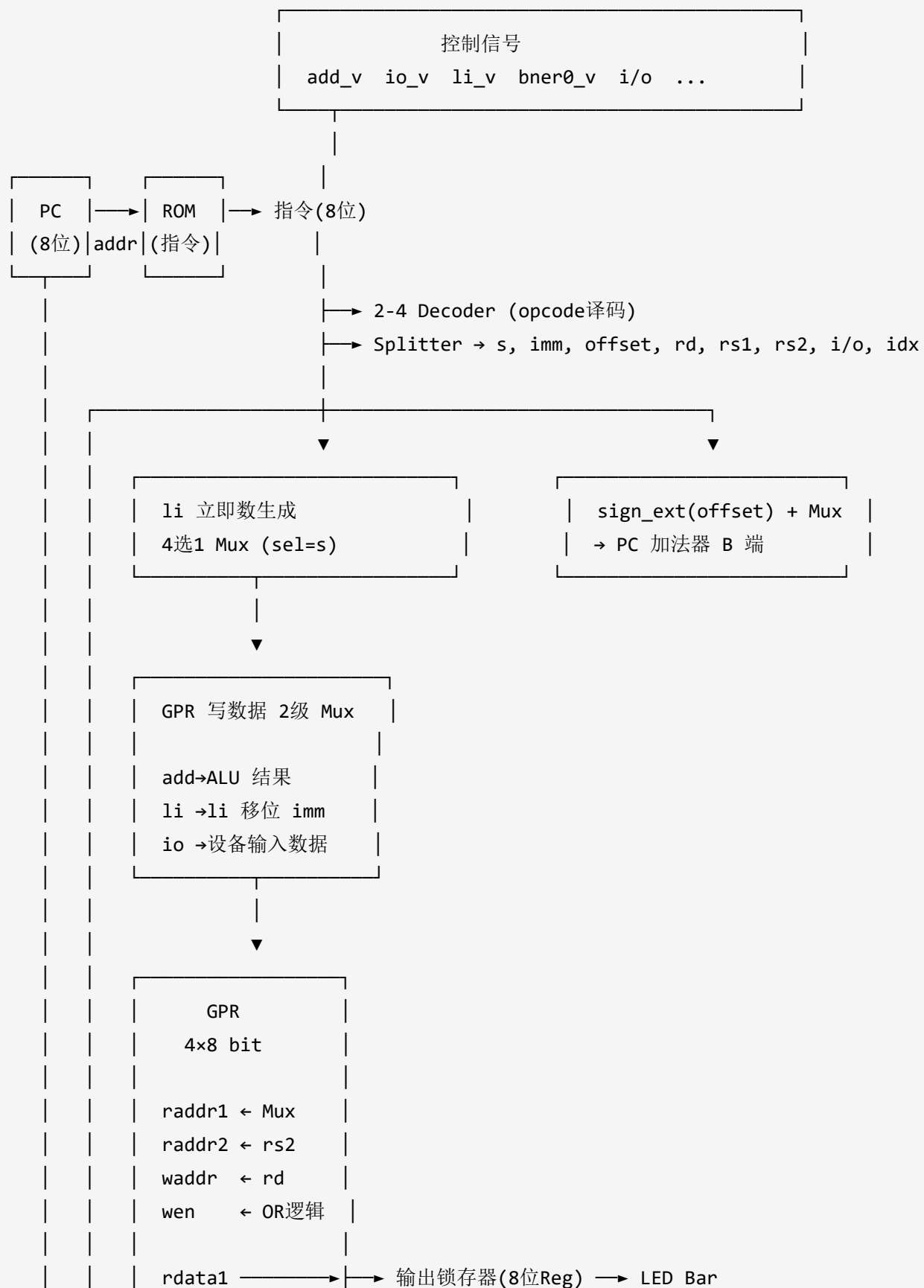
```
第1级: sel = bner0_valid
  数据0 = {add/io 的选择结果}
  数据1 = 0

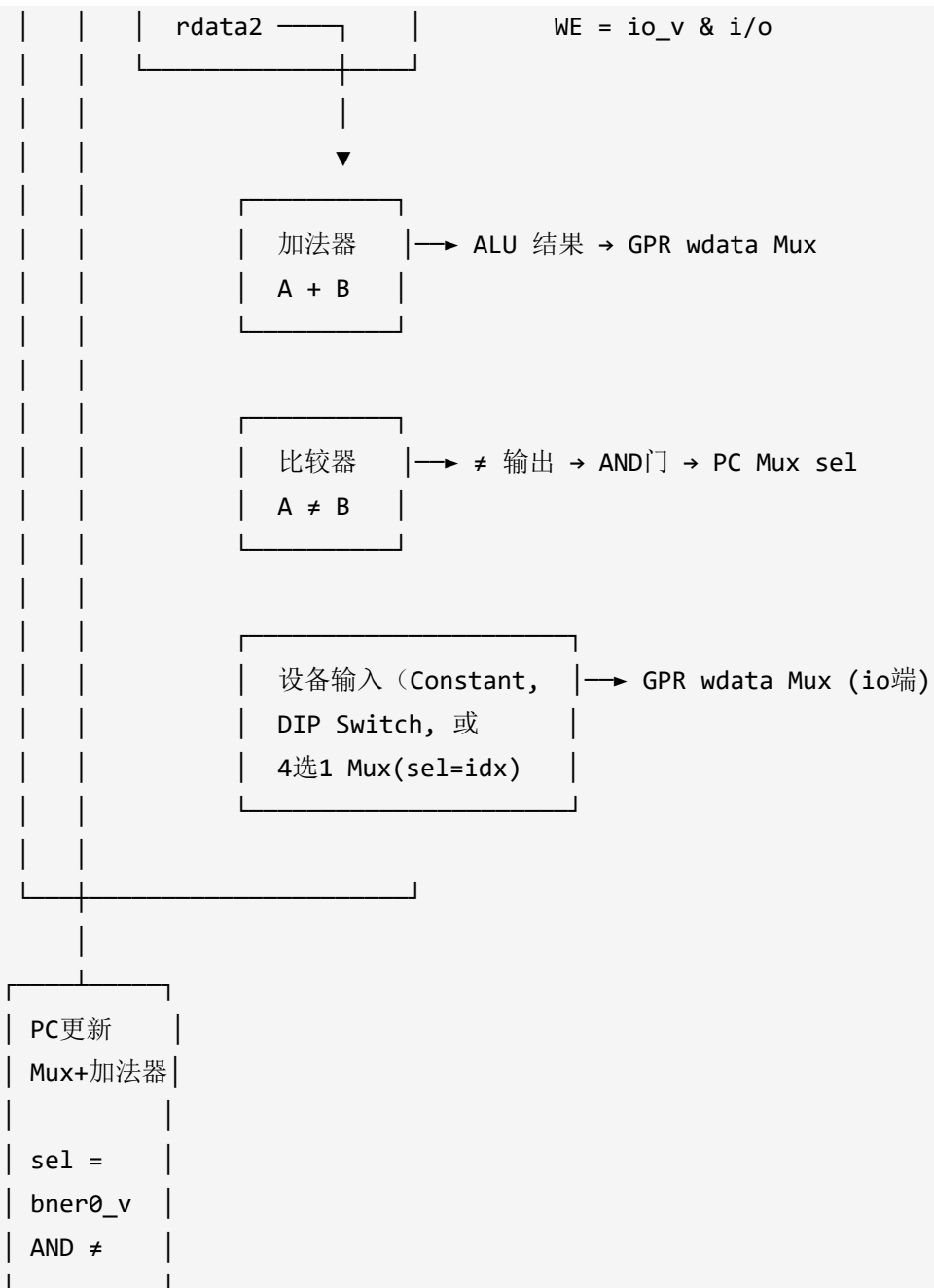
第2级: sel = io_valid AND i/o==1
  数据0 = rs1
  数据1 = rd
```

5.8 整体 GPR 读地址小结

指令	raddr1 (读口1)	raddr2 (读口2)
add	rs1	rs2
li	无关	无关
bner0	0 (<code>R[0]</code>)	rs2
io (输入)	无关	无关
io (输出)	rd (<code>R[rd]</code>)	无关

第六部分：完整电路总览





第七部分：测试程序

7.1 完整汇编程序（数列求和 + LED 输出）

```
# 计算 1+2+...+10 = 55 = 0b00110111
li r1, 00_00    # r1 = 0      (累加和)      s=00, imm=00
li r2, 00_01    # r2 = 1      (当前加数)    s=00, imm=01
li r3, 11_10    # r3 = 10     (循环上限)    s=11, imm=10
loop:
add r1, r1, r2   # r1 = r1 + r2
add r2, r2, 01   # r2 = r2 + 1      (01 即立即数1, s=00, imm=01)
bner0 -3, r3     # if r1 != r3 → 跳回 loop (偏移需根据实际电路计算!)
io r1, 1, 00     # 输出 r1 到设备 0 (LED)
```

⚠ **bner0 偏移量的计算取决于你的电路：**

- 如果 PC + sign_ext(offset) 中 PC 是指令地址（即取指时的 PC），从地址 5 跳回地址 3, offset = -2 = 1110
- 如果 PC 已经 +1（指向下一条指令），从地址 6 跳回地址 3, offset = -3 = 1101
- **建议先写一个简单跳转测试确定你的电路行为！**

7.2 机器码

地址	汇编	二进制机器码	十六进制
0	li r1, s=00, imm=00	10 01 00 00	0x90
1	li r2, s=00, imm=01	10 10 00 01	0xA1
2	li r3, s=11, imm=10	10 11 11 10	0xBE
3	add r1, r1, r2	00 01 01 10	0x16
4	add r2, r2, 01	00 10 10 01	0x29
5	bner0 offset=-3, r3	11 1101 11	0xF7
6	io r1, i/o=1, idx=00	01 01 1 00	0x54

7.3 将机器码写入 ROM

1. 右键 ROM → **Edit Contents**
2. 在十六进制编辑窗中按地址填入：

地址：值

00: 90

01: A1

02: BE

03: 16

04: 29

05: F7

06: 54

3. 点击 OK 保存

7.4 预期结果

- 程序运行完毕后：
 - $r1 = 55 = 0x37 = 00110111$
 - LED Bar 显示：第 0、1、2、4、5 颗 LED 亮，第 3、6、7 颗灭
 - 亮 → 灭 → 亮 → 亮 → 暗 → 亮 → 亮 → 暗 → 暗
 - （LED 排列：最右边是 bit 0，最左边是 bit 7）

第八部分：分步验证清单

按顺序逐步验证，每一步通过后再进行下一步。

验证 1：增强版 li 指令

- ☐ ROM 地址 0 写入 `li r0, s=00, imm=11`（机器码 0x83），运行后 $r0 = 0x03$ ？
- ☐ 改为 `li r0, s=11, imm=10`（0x8E），运行后 $r0 = 0x80$ ？
- ☐ 测试 4 种 s 值，确认立即数移位都正确

验证 2：增强版 bner0 指令

- ☐ 写最简单的测试： `li r1, 01; bner0 +1, r0; li r2, 02; li r3, 03`
 - 如果 bner0 正确跳转 ($R[0]=0, R[r0]=0$, 相等不跳) , r2 应该 = 0x02
 - 改成 bner0 +1, r1 ($R[0]=0, R[1]=0x01$, 不等) , 应跳过 `li r2, 02` , r2 保持 0
- ☐ 验证向后跳转 (负偏移) 也能正确工作

验证 3：io 输出到 LED

- ☐ ROM 写入： `li r0, AA; io r0, 1, 00` ($AA=0xAA=10101010$)
 - 运行后 LED 应交替亮灭
- ☐ 改为 `li r0, 55; io r0, 1, 00` ($0x55=01010101$)
 - LED 应显示相反的交替模式

验证 4：完整数列求和程序

- ☐ 运行第七部分的完整程序
- ☐ 检查 r1 最终 = 0x37 (55)
- ☐ 检查 LED 显示 00110111

验证 5：io 输入 (可选)

- ☐ 放一个 Constant = 0x3C 作为输入设备
- ☐ 写 `io r0, 0, 00; io r0, 1, 00` (先输入, 再输出)
- ☐ 检查 LED 是否与 Constant 一致

第九部分：常见问题排查

现象	可能原因	解决方法
LED 全不亮	LED Bar Input Format 不对	设为 One Wire
	输出锁存器 WE 没有正确触发	检查 <code>io_valid AND i/o==1</code> 的接线
	GPR rdata1 读的不是 rd	检查 GPR raddr1 的 Mux 是否正确

现象	可能原因	解决方法
	时钟没有启动	Ctrl+K 启动 Ticks
LED 全亮	锁存器输出悬空被拉到高电平	加一个 Reset 或初始化
	数据通路某处有错	加 Probe 逐步排查
LED 闪烁/乱跳	输出没加锁存器，直接连了组合逻辑	加上 Register 锁存输出
bner0 跳不到正确位置	offset 符号扩展错误	检查 Bit Extender 是否设为 Sign Extend
	offset 字段取错位	检查 Splitter 接线，offset 是 bits 5-2
	PC 基准不对（+1 与否）	确认电路用的是指令地址还是下一条地址
GPR 值被意外覆盖	io 输出时 wen 没关	检查 wen 逻辑：i/o=1 时 wen 必须为 0
	io 输入时数据通路选错	检查 GPR wdata Mux 选择逻辑
橙色线	位宽不匹配	检查所有 Splitter 和连接的位宽一致
蓝色线	信号悬空，未连接	补全接线
红色线	多驱动冲突	检查是否两个输出同时驱动同一根线

第十部分：进阶扩展（完成基本任务后可选）

10.1 支持多个 I/O 设备

用 `idx` 字段配合 2-4 Decoder 或 Demultiplexer 实现多设备：

输出（多设备）：

- 用 Demultiplexer（1→4），输入 = GPR rdata1，sel = idx，4 个输出各接一个 Register + 外设

输入（多设备）：

- 用 **4 选 1 Mux**, $sel = idx$, 4 个数据端接不同输入源

10.2 用数码管显示结果

- Input/Output → **7-Segment Display**
- 需要将 8 位结果转换为 BCD 码或直接显示十六进制
- 放置 2 个数码管, 一个显示十位, 一个显示个位

10.3 用 Button 控制输入

- Input/Output → **Button**, 按一下输入一个值
- 配合 D Flip-Flop 做去抖动

完成 F6 后, 你就拥有了一个可以和外界交互的简单处理器! LED 亮灭的那一刻, 说明你亲手设计的 CPU 正在和物理世界对话。